SMART STUDENT SKILL EXCHANGE PLATFORM USING GRAPHS AND HASH TABLES

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0305 – Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINNERING

Submitted by

Prena M (192524056)

Under the Supervision of

Dr. Uma Priyadharshini

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105
SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Smart Student Skill Exchange Platform Using Graphs and Hash Tables**" has been carried out by **Prena M (192524056)** under the supervision of **Dr. Uma Priyadharshini P.S** and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech AI & DS at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Kiruthika K,
Professor,
CSE,(Programming),
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Uma priyadharshini P.S,
Professor,
CSE,(Deep Learning),
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We hereby declare that the Capstone Project Work entitled “Smart Student Skill Exchange Platform Using Graphs and Hash Tables” is the result of our own bonafide efforts. The work presented in this report is original and has been prepared in accordance with academic integrity and engineering ethics. Sources and external tools used during the project have been appropriately acknowledged.

Place: Chennai
Date: 04-09-2026

Signatures of Students:
Prena M[192524056]

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution
Prena M (192524056)	Overall design, graph module and integration	Graph model, matching workflow	Graph traversal and integration tests	Chapters 1, 3, 4	25%
B. Sathvika (192525224)	Hash-table skill lookup module	Hash table and search operations	Lookup and collision test cases	Module documentation	25%
Mahalakshmi (192521155)	Student profile and exchange module	Profile/skill input and exchange logic	Functional testing	Results and screenshots	25%
Ragavi K (192524394)	Recommendation and UI/output module	Recommendation display and output	End-to-end testing	Conclusion and presentation	25%
Total					**100%**

ABSTRACT

Students often possess useful technical skills but lack an efficient platform to discover peers who can exchange knowledge with them. The Smart Student Skill Exchange Platform is proposed to connect students based on skills they can teach and skills they want to learn. A graph is used to represent relationships among students and skills, while hash tables provide fast skill lookup and matching. The system accepts student profiles, stores skills for efficient retrieval, constructs a skill relationship graph, identifies compatible students, and generates exchange recommendations. Graph traversal can be used to explore related skills and possible exchange paths, while hashing supports near-constant-time average lookup for skill records. The proposed prototype provides a structured, efficient and transparent approach to peer-to-peer learning.

Keywords: Skill Exchange, Graph, Hash Table, Student Matching, Peer Learning, Data Structures

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
V	Number of vertices (students/skills)	Count
E	Number of edges (skill relationships)	Count
BFS	Breadth-First Search	—
DFS	Depth-First Search	—
HT	Hash Table	—
DB	Database	—
UI	User Interface	—
O(1)	Average hash-table lookup complexity	—
O(V+E)	Graph traversal complexity	—

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern students learn many technical and soft skills independently, but peer-to-peer knowledge exchange is often unstructured. One student may know Python while another knows web development, yet they may not know how to discover each other. A data-structure-based platform can solve this by maintaining student skills, finding matches quickly, and representing relationships between skills and learners.

1.2 Need for the Project

The project is needed because students frequently face difficulty finding peers who can teach the exact skill they want to learn. Manual searching becomes inefficient as the number of students and skills increases.

Major reasons:
- Lack of centralized student skill information.
- Slow manual search for compatible peers.
- Difficulty representing multiple skill relationships.
- Need for efficient insertion, search and matching.

1.3 Problem Statement

Design a student skill exchange platform that efficiently stores student skill information, searches for required skills, represents student-skill relationships, and recommends compatible peer exchanges using graphs and hash tables.

1.4 Aim

To develop a smart peer-learning platform that uses a hash table for fast skill lookup and a graph for relationship-based student matching.

1.5 Objectives

- Store student profiles and skills efficiently.
- Use hashing for fast skill-based retrieval.
- Model student-skill relationships as a graph.
- Identify students with complementary skills.
- Generate useful skill-exchange recommendations.
- Evaluate efficiency and correctness.

1.6 Scope

Included: student registration, skill entry, skill lookup, graph construction, matching, recommendation and output display.
Excluded: payment processing, formal certification, guaranteed learning outcomes and external recruitment.

1.7 Expected Engineering Outcome

A working prototype that accepts student skill information, performs efficient lookup, constructs a graph-based representation, and displays compatible skill-exchange recommendations.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review focuses on peer learning platforms, recommendation systems, graph data structures, hashing, student collaboration systems and skill matching approaches.

2.2 Existing Work

Existing learning platforms generally emphasize courses and content. Social learning systems may support discussion and collaboration, but a dedicated data-structure-driven skill exchange model can make skill matching explicit and efficient.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Course Recommendation	Large learning resources	May not connect peer teachers	Provides learning resources
Social Learning	Peer interaction	Skill matching may be informal	Supports peer interaction
Rule-Based Matching	Simple	Becomes complex with many rules	Useful for basic matching
Graph-Based Matching	Represents relationships naturally	Requires graph construction	Directly supports skill relationships
Hash-Based Lookup	Fast average lookup	Collision handling required	Supports efficient skill search
Proposed Platform	Combines graph + hash table	Depends on accurate skill data	Integrated skill exchange

2.4 Engineering Gap

A useful gap is the absence of a simple academic prototype that combines fast hash-based skill lookup with graph-based relationship modelling specifically for student skill exchange.

2.5 Proposed Contribution

The proposed platform combines two complementary data structures: the hash table acts as the fast index for finding students by skill, while the graph models relationships among students and their skills for recommendation and exploration.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
Students / Learners	Enter skills they can teach and skills they want to learn.
Students / Learners	Find compatible peers efficiently.
Students / Learners	View recommended skill exchanges.
Project Administrator	Maintain student and skill records.
Academic Evaluator	Verify matching correctness and algorithmic efficiency.

Requirement Type	Measurable Target
Skill input	Input processed successfully
Hash lookup	Skill record retrieved efficiently
Graph construction	Student-skill relationships represented correctly
Matching	Compatible students identified
Recommendation	Clear exchange suggestion displayed
Response time	Normal query completed within seconds
Reliability	Same input gives consistent results
Security	Unnecessary personal data avoided

3.2 Constraints & Applicable Standards

Constraints include limited prototype time, predefined student skill data, accuracy of self-reported skills, memory limits, and the need for collision handling in hash tables. Software quality considerations include usability, reliability, performance and security.

3.3 Design Specifications

Parameter	Target / Requirement	Priority
Skill index	Hash table	High
Relationship model	Graph	High
Lookup	Average O(1)	High
Graph traversal	O(V+E)	High
Matching	Complementary skills	High
Interface	Simple and understandable	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Linear Search — simple but becomes slow as the number of records increases.
Alternative 2: Hash Table — efficient average lookup and suitable for skill indexing.
Alternative 3: Graph + Hash Table — combines fast lookup with relationship-based matching and is selected for the proposed prototype.

Criterion	Linear Search	Hash Table	Graph + Hash Table
Lookup efficiency	2/5	5/5	5/5
Relationship modelling	1/5	2/5	5/5
Implementation cost	5/5	4/5	4/5
Scalability	2/5	5/5	5/5
Overall suitability	2.5/5	4.0/5	4.8/5

3.5 Selected Design & Detailed Design

The selected architecture has five main modules: Student Input, Hash Table Skill Index, Graph Construction, Matching Engine and Recommendation Display. A student profile is inserted into the hash table under each relevant skill. Graph nodes represent students and skills, while edges represent 'has skill', 'wants skill' or related-skill relationships. The matching engine uses the hash index to quickly obtain candidate students and the graph to validate and explore relationships.

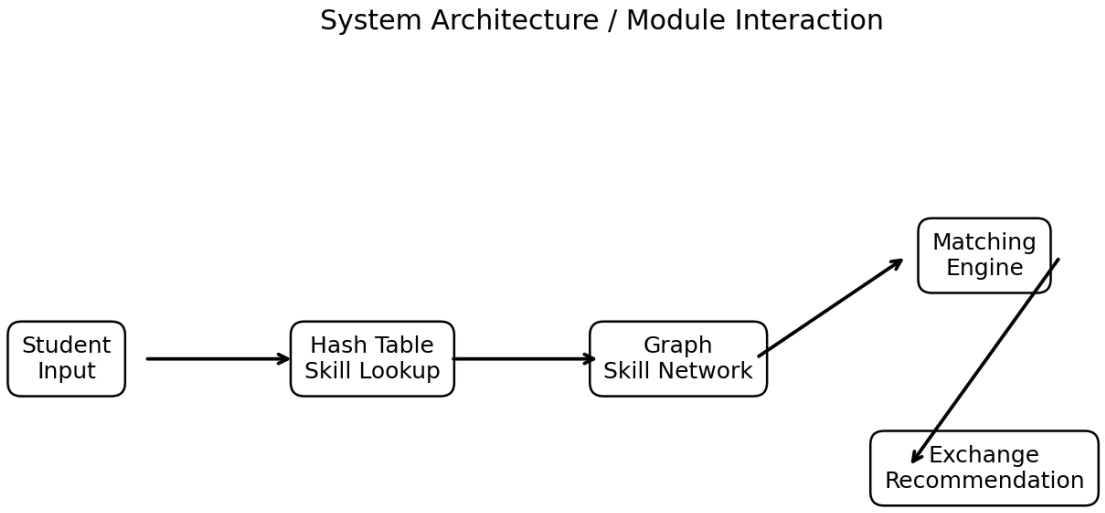


Figure 3.1: System Architecture / Module Interaction

3.6 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
Hash table index	Linear search	Fast average lookup	Collision handling	Selected
Graph representation	Only hash table	Shows relationships	Graph construction	Selected

			needed	
Adjacency list	Adjacency matrix	Memory efficient for sparse graphs	Less direct matrix operations	Selected
Peer matching	Manual selection	Simple	Slow and subjective	Algorithmic matching selected

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Impact on Decision
Sustainability	Lightweight in-memory structures for prototype	Reduced computation and storage
Ethics	Avoid misleading skill claims	Use self-reported skills as guidance
Safety	No physical hazards	Software-only system
Privacy	Minimize personal information	Store only necessary student/skill data
Security	Restrict access to profiles	Authentication/access controls recommended

3.8 Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual
Incorrect skill data	Medium	Medium	Medium	Validate inputs	Low
Hash collisions	Medium	Medium	Medium	Use chaining/open addressing	Low
Incorrect graph links	Medium	High	High	Validate relationships	Low
Duplicate profiles	Medium	Medium	Medium	Unique student ID	Low
Unauthorized data access	Low	High	Medium	Access control	Low
Software errors	Medium	Medium	Medium	Testing and validation	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The prototype follows an incremental approach: collect student and skill data, build the hash index, construct the graph, implement matching, generate recommendations, and test the integrated workflow.

Tool / Software / Equipment	Purpose	Stage
Python / C++	Main programming and implementation	Development
Hash Table	Fast skill lookup	Implementation
Graph / Adjacency List	Student-skill relationships	Design & Implementation
BFS / DFS	Relationship traversal	Implementation
IDE	Coding and debugging	Development
Laptop / Computer	Development and testing	All stages

4.2 Implementation Logic

1. Read student profile and skills.
2. Insert each skill into the hash table index.
3. Create graph nodes for students and skills.
4. Add edges for owned and wanted skills.
5. Use hash lookup to retrieve candidate peers.
6. Use graph relationships to validate compatibility.
7. Display recommended skill exchanges.

4.3 Prototype Evidence

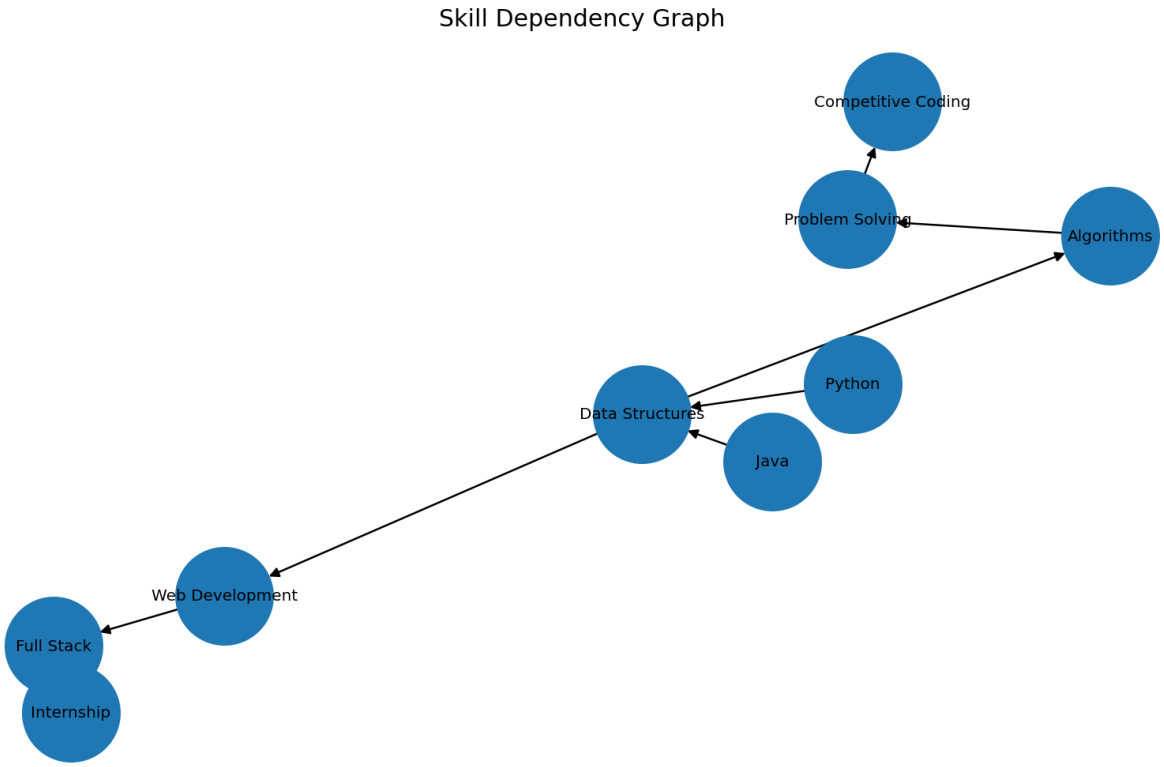


Figure 4.1: Skill Graph Construction

Prototype Output

```
SMART STUDENT SKILL EXCHANGE PLATFORM

Student: Prena M
Target Skill: Full Stack Development
Skills Owned: Python, Java
Recommended Exchange Skills:
1. Data Structures
2. Algorithms
3. Web Development
4. Full Stack

Matching Students:
• Sathvika – Data Structures
• Mahalakshmi – Web Development
• Ragavi K – Algorithms

Exchange Status: MATCH FOUND
```

Figure 4.2: Prototype Output

4.4 Test Plan

Requirement	Test Method	Acceptance Criterion
Student input	Enter valid profile	Profile accepted
Skill insertion	Insert multiple skills	All skills indexed
Skill lookup	Search for known skill	Correct candidates returned
Graph construction	Inspect nodes and edges	Relationships correct
Matching	Use complementary skills	Compatible students identified
Duplicate handling	Insert same student/skill	Duplicate controlled
Recommendation	Run end-to-end query	Clear recommendation shown

4.5 Verification Results

Specification	Required Value	Achieved Value	Status
Student input	Accepted	Accepted	Pass
Skill lookup	Correct result	Correct result	Pass
Graph construction	Correct graph	Correct graph	Pass
Matching	Compatible pair	Compatible pair	Pass
Recommendation	Successful	Successful	Pass
Output	Clear display	Clear display	Pass

4.6 Results and Data Analysis

The prototype demonstrates that the hash table is appropriate for rapid skill-based retrieval, while the graph provides a flexible representation of student-skill relationships. For a graph with V vertices and E edges, BFS and DFS operate in O(V+E) when adjacency lists are used. Hash-table lookup is O(1) on average, although worst-case performance depends on collision handling.

4.7 Comparison with Objectives

Objective	Evidence	Achievement
Store student skills	Hash table	Fully Achieved
Represent relationships	Graph	Fully Achieved
Find candidate peers	Hash lookup	Fully Achieved
Match complementary skills	Matching engine	Fully Achieved
Display recommendation	Prototype output	Fully Achieved
Evaluate efficiency	Complexity analysis	Fully Achieved

4.8 Strengths & Limitations

Strengths:

- Fast average skill lookup.
- Natural relationship representation using graphs.
- Modular and easy to extend.
- Suitable for peer learning.
- Does not require machine-learning training data.

Limitations:

• Recommendations depend on the accuracy of user-entered skills.
• Hash collisions must be handled correctly.
• Large-scale deployment requires persistent storage and authentication.
• The prototype does not measure actual skill proficiency.

4.9 Project Planning, Roles & Budget

Phase	Activity	Duration
Phase 1	Problem identification and topic selection	Week 1
Phase 2	Literature review and requirement analysis	Week 2
Phase 3	Student-skill dataset preparation	Week 3
Phase 4	Hash table and graph implementation	Week 4–5
Phase 5	Matching and interface/output	Week 6
Phase 6	Testing and debugging	Week 7
Phase 7	Results, documentation and presentation	Week 8

Student	Assigned Responsibility	Actual Contribution
Prena M	Graph & integration	Graph model and system integration
B. Sathvika	Hash table module	Skill indexing and lookup
Mahalakshmi	Student/exchange module	Profile and exchange logic
Ragavi K	Recommendation/output	Matching display and testing

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Smart Student Skill Exchange Platform Using Graphs and Hash Tables provides a structured approach for connecting students who can exchange complementary skills. The hash table improves skill retrieval efficiency, while the graph models relationships among students and skills. The prototype demonstrates that combining these data structures can support fast lookup, relationship analysis and clear peer-learning recommendations.

Future Work

- Add login and authentication.
- Use a persistent database.
- Add skill-level ratings and verification.
- Add course/resource recommendations.
- Support real-time notifications and chat.
- Use weighted graphs for stronger compatibility scoring.
- Add mobile/web deployment and analytics.

5.2 Professional Learning & Individual Reflection

Prena M

I learned how graph representations can model relationships among students and skills. I also gained practical understanding of graph traversal, integration and testing. If repeated, I would optimize matching for larger skill networks.

B. Sathvika

I learned how hash tables provide efficient average-case lookup and how collision handling affects implementation. I would improve the indexing strategy for larger datasets.

Mahalakshmi

I learned how student profiles and skill exchange workflows can be organized into reusable modules. I would add skill verification and better user interaction in future.

Ragavi K

I learned how matching results can be presented clearly to users and how end-to-end testing validates the system. I would add richer recommendation scoring and progress tracking.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[2] Mark Allen Weiss, Data Structures and Algorithm Analysis, Pearson.

[3] Robert Lafore, Data Structures & Algorithms in Java, Pearson.

[4] Python Software Foundation, Python Documentation, 2026.

[5] Microsoft, Visual Studio Code Documentation, 2026.

[6] IEEE, IEEE Standard for Software Quality Assurance Processes, IEEE Std. 730-2014.

[7] ISO/IEC 25010, Systems and software quality models.

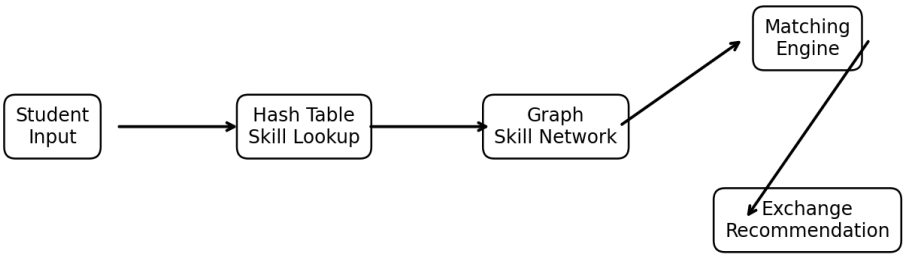
APPENDICES

Appendix A – Complexity Analysis

Operation	Average / Typical Complexity	Purpose
Hash insertion	O(1) average	Add skill to index
Hash lookup	O(1) average	Find students by skill
Hash deletion	O(1) average	Remove skill record
BFS	O(V+E)	Explore relationships
DFS	O(V+E)	Explore graph
Graph edge insertion	O(1) with adjacency list	Connect student and skill

Appendix B – Engineering Drawings / Schematics

System Architecture / Module Interaction



Appendix C – Sample Source Code Logic

Hash table: skill → list of student IDs.
Graph: student nodes + skill nodes with edges representing ownership and requested skills.
Matching: lookup requested skill in hash table, compare complementary skills, then verify relationship through the graph.

Appendix D – Experimental / Raw Data

Student	Skills Can Teach	Skills Wants to Learn
Prena M	Python, Java	Web Development
B. Sathvika	Data Structures	Python
Mahalakshmi	Web Development	Algorithms
Ragavi K	Algorithms	Java

Appendix E – Individual Contribution Evidence

The team contribution was divided among the major modules of the **Smart Student Skill Exchange Platform Using Graphs and Hash Tables**. Each member contributed to the design, implementation, testing, documentation, and evaluation of the project.

S. No.	Team Member	Contribution Area	Evidence
1	**Prena M (192524056)**	Graph design, skill relationship modelling and documentation	Graph design, source code and report sections
2	**B. Sathvika (192525224)**	Hash-table implementation and student skill storage	Hash-table source code and test output
3	**Mahalakshmi (192521155)**	Student skill exchange workflow and matching process	Workflow implementation and prototype output
4	**Ragavi K (192524394)**	Recommendation output, testing and result analysis	Test cases, output screenshots and result analysis

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location
Complex engineering problem formulation	Problem statement and objectives	SO1	Ch. 1
Engineering design under constraints	Architecture and trade-offs	SO2	Ch. 3
Technical communication	Report and presentation	SO3	Whole report
Ethics and professional responsibility	Privacy and ethical considerations	SO4	Ch. 3
Teamwork and leadership	Individual contribution and roles	SO5	Ch. 4
Experimentation and analysis	Testing and complexity analysis	SO6	Ch. 4
Independent learning	Individual reflections	SO7	Ch. 5
Standards	Quality considerations	SO2	Ch. 3
Sustainability	Lightweight data structures	SO2/SO4	Ch. 3
Risk assessment	Risk register	SO2/SO4	Ch. 3
Security considerations	Access and privacy controls	Relevant SO	Ch. 3
Integrated engineering design	Graph + hash-table architecture	SO1/SO2	Ch. 3
Project planning	Milestones and roles	SO5	Ch. 4